

OPERATING SYSTEMS AND SYSTEMS PROGRAMMING (25CS2104E)

2026–27, Term-I

Project Problem Statement Submission Form

Section / Team Details

Programme / Year: B.Tech – II Year

Section No.: 13

Team No.: 20

Project Title: Linux Signal-Based Process Communication and Control System

Team Members

Roll Number	Student Name	Signature
2520090226	Lahari Namala	
2520090236	K. Keathana Vikashini	
2520080024	P.Bhagya sri	
2520090234	C.Poojitha	

1. Abstract

Signals are one of the oldest and simplest ways of doing Inter-Process Communication (IPC) in Linux. The operating system uses them to inform a process that some event has occurred, such as a request to stop, an interrupt from the keyboard, or a message sent by another process. In this project we are building a Linux Signal-Based Process Communication and Control System in C, which shows how one parent process can communicate with and control several child processes using signals. The parent creates the child processes using `fork()`, and every child installs its own signal handlers using `sigaction()`. The parent can then send signals such as `SIGUSR1`, `SIGUSR2`, `SIGSTOP`, `SIGCONT` and `SIGTERM` to a particular child using `kill()`, so that the child pauses its work, resumes it, prints its current status, or exits cleanly instead of being killed suddenly. We also use `sigprocmask()` to block signals for a short time while a child is inside a critical section, so that its data is not left half-updated, and `sigsuspend()/pause()` so that a child can wait for a signal without wasting CPU time. A simple menu-driven controller lets the user send signals to any child by its PID and see how it responds, and the process state is also verified from the `/proc` filesystem. The expected outcome is a working command-line tool that demonstrates process control using signals, along with a report on the results.

2. Problem Statement

In a multi-process Linux application, processes often need a lightweight way of being informed about an event or being told to change their behaviour, without the overhead of setting up pipes, message queues or shared memory. Signals provide exactly this kind of asynchronous notification at the operating system level, but they are also easy to use incorrectly. Handlers are often written using unsafe functions, race conditions occur between the moment a signal is delivered and the moment it is handled, and a signal that arrives while the same signal is already pending is simply lost. This project addresses the problem of communicating reliably between a parent process and its child processes, and of controlling their execution (pause, resume,

status report, terminate) using Linux signals, while handling signal masking, critical sections and race conditions correctly.

3. Objectives

1. To design and implement a multi-process system in which a parent process creates and manages several child processes using `fork()`, `exec()` and `waitpid()`.
2. To write custom signal handlers using `sigaction()` for the signals `SIGUSR1`, `SIGUSR2`, `SIGTERM`, `SIGCONT` and `SIGSTOP`, so that the parent can control the behaviour of each child.
3. To use signal masking with `sigprocmask()` and `sigsuspend()` in order to protect critical sections and avoid race conditions during signal handling.
4. To build a controller program that sends signals to a chosen process by its PID, displays the status of all child processes, and logs every signal event with a timestamp.

4. Proposed Methodology

The system will be implemented in C on Ubuntu Linux. A parent (controller) process will use `fork()` to create a fixed number of child worker processes. Each child installs its signal handlers using `sigaction()` and then enters a loop in which it performs some simulated work and waits to be interrupted. Based on the command typed by the user (pause, resume, status, terminate), the parent sends the matching signal to the selected child using `kill(pid, signal)`. `SIGUSR1` and `SIGUSR2` are used as application-defined control messages, `SIGSTOP` and `SIGCONT` are used to suspend and resume a child, and `SIGTERM` is used for graceful termination so that the child can free its resources before exiting.

Inside the work loop of a child, the critical section is protected by blocking signals temporarily with `sigprocmask()`, and `sigsuspend()` is used to unblock the signals and wait for the next one atomically, which avoids the race condition that occurs when a signal arrives just before the process starts waiting. The parent keeps a table of all children (PID, current state, last signal received) and writes every event to a log file with a timestamp. The state shown by the program is cross-checked with the information read from `/proc/[pid]/status`, and the system is finally tested with several child processes running at the same time to confirm that the signals are delivered and handled correctly.

5. Operating Systems Concepts / Linux APIs Used

OS Concept / Linux API / System Call	Purpose in the Project
Process creation: <code>fork()</code> , <code>exec()</code> , <code>waitpid()</code>	Create the child worker processes and clean up terminated children
<code>getpid()</code> , <code>getppid()</code>	Identify each process and display the parent-child relationship
Signal handling: <code>signal()</code> , <code>sigaction()</code>	Install custom handlers for the signals used as control messages
<code>kill()</code> , <code>raise()</code>	Send signals from the parent to a particular child process by PID
<code>sigprocmask()</code> , <code>sigsuspend()</code> , <code>pause()</code>	Block and unblock signals to protect critical sections and wait safely
<code>SIGUSR1</code> , <code>SIGUSR2</code> , <code>SIGTERM</code> , <code>SIGCONT</code> , <code>SIGSTOP</code> , <code>SIGCHLD</code>	Signals used for pause, resume, status report and graceful termination

OS Concept / Linux API / System Call	Purpose in the Project
Process states / zombie and orphan processes	Observe how a child moves between running, stopped and terminated states
Critical section and race conditions	Ensure shared data is not corrupted when a signal arrives during an update
/proc/[pid]/status	Independently verify the process state reported by our program

6. Tools / Platforms / Software Used

Tool / Platform / Software	Purpose
Linux / Ubuntu	Development and demonstration environment
C (GCC compiler)	Implementation language for all process and signal-handling code
GDB / strace	Debugging the program and tracing signal delivery and system calls
ps, top, kill (shell commands)	Observing process states and testing signals from the terminal
Git / GitHub	Version control and source code management
Visual Studio Code	Code editing

7. Expected Outcome

A working command-line Linux application in which a parent process is able to start, pause, resume, get the status of, and terminate multiple child processes using POSIX signals, with correct use of signal handlers, signal masking and protection of critical sections. The controller will display the live status of all child processes and write every signal event to a log file with a timestamp.

A project report presenting the design of the system, the Linux/POSIX system calls used, screenshots of the output, and the test results obtained with different numbers of child processes, including the cases where a repeated signal was lost because the same signal was already pending.

8. GitHub Repository

GitHub Repository Link: https://github.com/lahari2007namala-creator/2520090226_OSSP/tree/main/PROJECT

Repository Created: Yes

☒ Yes ☐ No

Faculty Name and Signature

Faculty Name: _____

Signature: _____

Remarks: _____

Faculty Use

Project Approved: _____

☐ Yes ☐ No

Date: _____

Faculty Signature: _____